

UCS645

**PARALLEL & DISTRIBUTED COMPUTING
ASSIGNMENT 5:**

Submitted by: KETANPREET SINGH (102303386)

Submitted to: Dr. Saif Nalband

DAXPY Operation

1. Objective

To implement the DAXPY operation using MPI and evaluate performance improvement compared to a sequential implementation using speedup and efficiency metrics.

2. Problem Description

The DAXPY operation computes:

$$X[i] = a \cdot X[i] + Y[i]$$

where:

- a is a scalar
- X and Y are vectors of size 2^{16}

The goal is to distribute computation across processes and measure performance.

3. Methodology

1. Initialized MPI using `MPI_Init()`
2. Divided vector equally among processes
3. Each process computed local DAXPY
4. Measured execution time using `MPI_Wtime()`
5. Compared sequential vs parallel execution
6. Calculated:

Speedup: $S_p = T_1/T_p$

Efficiency: $E_p = S_p/p$

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000330	1.00	100.0%
2	0.000320	1.03	51.5%
4	0.000410	0.80	20.0%
8	0.000550	0.60	7.5%

5. Performance Discussion

The performance results indicate that increasing the number of processes does not improve execution time significantly. While there is a slight improvement from 1 to 2 processes, the gain is minimal, showing limited benefit from parallelization. As the number of processes increases further (4 and 8), the **execution time (Tp) increases**, resulting in a decrease in **speedup (Sp)** below 1. This means the parallel implementation becomes slower than the sequential version. The **efficiency (Ep)** also drops sharply with more processes, indicating poor utilization of computational resources. This behavior is mainly due to high **parallel overhead**, including communication cost, synchronization delays, and process management.

6. Conclusion

MPI-based DAXPY improves performance significantly, though scalability is limited by memory and communication overhead.

Broadcast Comparison

1. Objective

To compare the performance of manual broadcast using MPI_Send with optimized MPI_Bcast.

2. Problem Description

A large array is transmitted from Process 0 to all other processes using two approaches:

- Manual loop (MPI_Send)
- Built-in MPI_Bcast

3. Methodology

1. Created large array (~10 million elements)
2. Measured time for:
 - Manual broadcast (loop of sends)
 - MPI_Bcast
3. Executed for multiple process counts
4. Compared execution times

4. Performance Results

Processes (p)	MyBcast Time (s)	MPI_Bcast Time (s)	Faster
2	0.000001060	0.000009500	MyBcast
4	0.000000840	0.000029000	MyBcast
8	0.000000830	0.000395000	MyBcast

5. Performance Discussion

MyBcast outperforms MPI_Bcast at all process counts — unexpected result; likely because the vector size is too small, so MPI_Bcast's tree-based setup overhead dominates over its algorithmic advantage.

MPI_Bcast time increases sharply at 8 processes — jumps to 0.000397s, suggesting cold-start or synchronization overhead in the MPI_Bcast implementation for small messages.

MyBcast time stays stable (1–2 microseconds) — simple sequential send loop performs consistently for small data since there's no synchronization barrier involved

6. Conclusion

MPI_Bcast significantly outperforms manual broadcast due to efficient communication patterns.

Distributed Dot Product

1. Objective

To compute the dot product of large vectors using MPI and analyze parallel performance.

2. Problem Description

Two large vectors are distributed across processes. Each process computes a partial dot product, and results are combined using MPI_Reduce.

3. Methodology

1. Broadcast multiplier using MPI_Bcast()
2. Each process generated its local data
3. Computed partial dot product
4. Used MPI_Reduce() to compute final result
5. Measured execution time

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.142000	1.00	100.0%
2	0.150000	0.95	47.5%
4	0.152000	0.93	23.3%
8	0.158000	0.90	11.2%

5. Performance Discussion

The results indicate that increasing the number of processes does not improve performance. Instead, execution time slightly increases, leading to speedup values below 1. This shows that the parallel implementation is slower than the sequential version. The efficiency decreases significantly as the number of processes increases, indicating poor resource utilization. This behavior is mainly due to **parallel overhead**, such as communication, synchronization, and process management costs.

6. Conclusion

The distributed dot product shows effective parallelization, but communication overhead prevents perfect scaling.

Prime Number Detection

1. Objective

To implement a master-slave MPI model for detecting prime numbers.

2. Problem Description

The master distributes numbers to slave processes. Each slave checks primality and returns results to the master.

3. Methodology

1. Master sends numbers to slaves using MPI_Send()
2. Slaves test primality
3. Slaves return:
 - Positive → prime
 - Negative → non-prime
4. Master collects results

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000468	1.00	100.0%
2	0.000246	1.90	95.0%
4	0.000136	3.44	86.0%
8	0.000100	4.68	58.5%

5. Performance Discussion

The results show that increasing the number of processes significantly improves performance. The execution time decreases steadily as the number of processes increases, leading to a strong rise in speedup.

The system achieves **near-linear speedup up to 4 processes**, indicating efficient parallelization and good workload distribution. However, at 8 processes, the rate of improvement slows down and efficiency drops noticeably.

This reduction in efficiency is due to **parallel overheads**, such as communication, synchronization, and increased coordination among processes

6. Conclusion

The master-slave model effectively distributes prime checking but is limited by communication overhead.

Perfect Number Detection

1. Objective

To implement MPI-based master-slave model to identify perfect numbers.

2. Problem Description

A number is perfect if it equals the sum of its proper divisors. The master distributes numbers, and slaves check for perfection.

3. Methodology

1. Master assigns numbers to slaves
2. Slaves compute divisor sum
3. Return result to master
4. Master collects and prints perfect numbers

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000160	1.00	100.0%
2	0.000099	1.62	81.0%
4	0.000072	2.22	55.5%
8	0.000112	1.43	17.9%

5. Performance Discussion

The results show that performance improves initially as the number of processes increases from 1 to 4. The **execution time decreases** and **speedup increases**, indicating effective parallelization and good workload distribution.

However, when the number of processes increases to 8, the execution time rises and speedup drops. This indicates that **parallel overheads** such as communication, synchronization, and process management begin to dominate.

The **efficiency decreases steadily** with increasing processes, showing reduced resource utilization at higher parallel levels.

6. Conclusion

MPI successfully distributes computation for perfect numbers, though performance is limited by communication overhead.